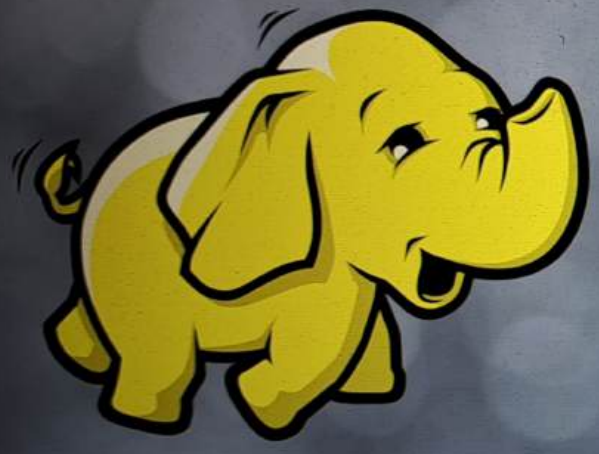


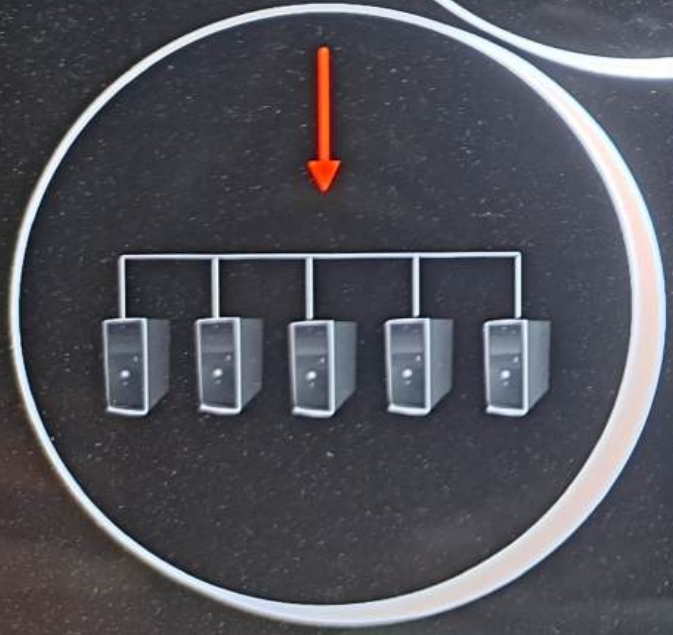
Hadoop Platform

YARN - Cluster Operating System

HDFS - Distributed Storage

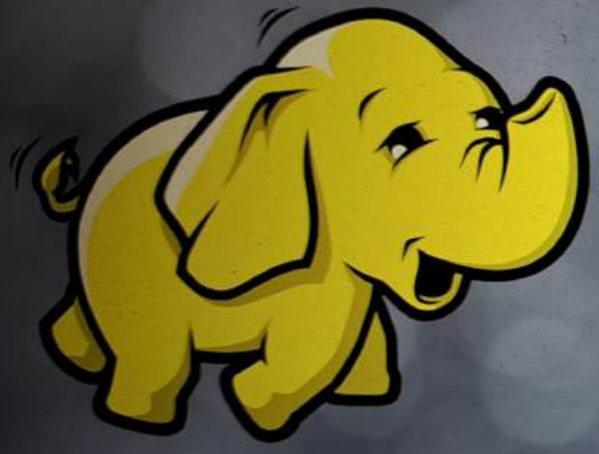
Map/Reduce – Distributed Computing

Not usable without Computer Operating System



Not usable without Cluster Operating System

udemy



Hadoop Platform

YARN - Cluster Operating System

HDFS - Distributed Storage

Map/Reduce – Distributed Computing

HIVE

APACHE HBASE

sqoop

OOZIE

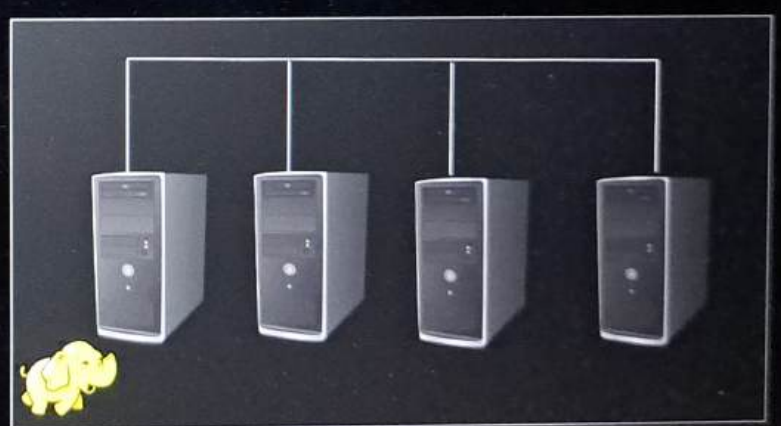
Databases and Hadoop

Database

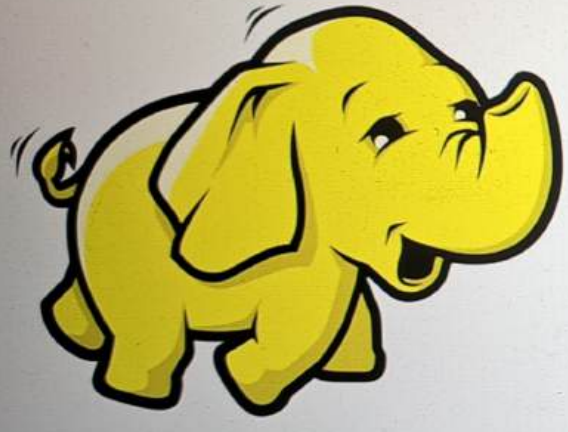
1. Data storage
2. SQL Query language
3. Scripting language
4. Programming languages Interface

Hadoop

1. Data storage
2. Hive SQL Query language
3. Pig Scripting language
4. Programming languages Interface

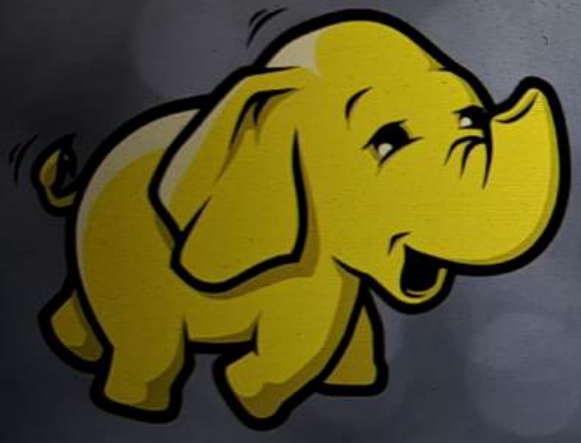


udemy



In this lecture

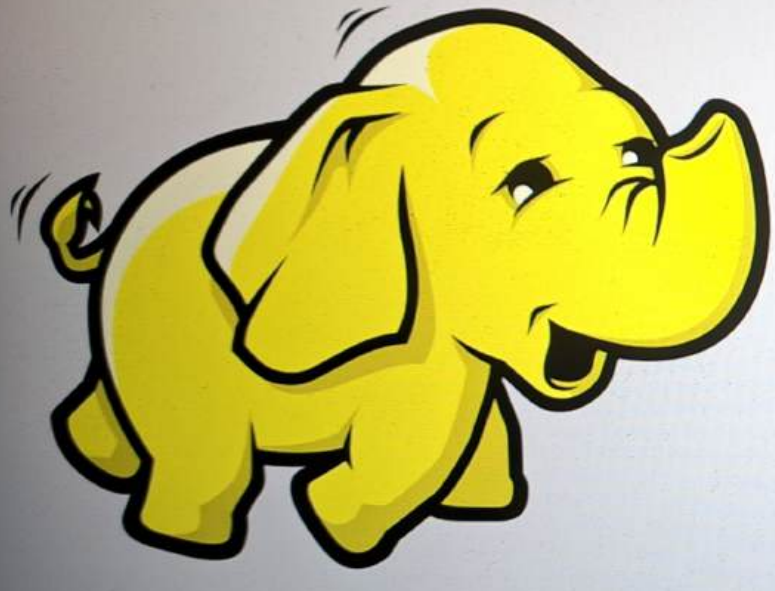
- What is Hadoop and how it works?
- Where did it come from?
- Challenges with Hadoop
- Entering Apache Spark
- Data Lake Vs Lakehouse



Hadoop Platform

Hadoop is a distributed data processing platform that offers the following core capabilities.

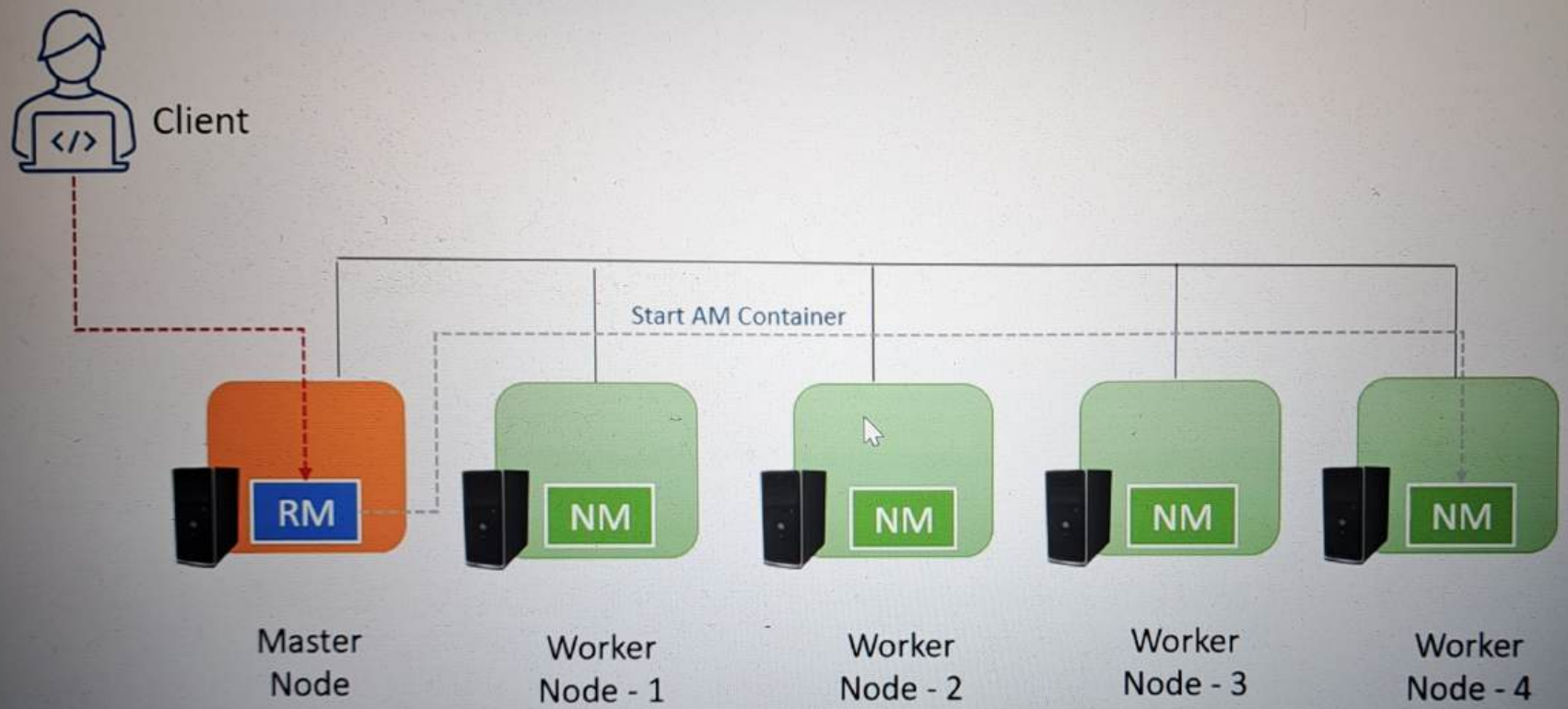
- YARN - Cluster Resource Manager
- HDFS - Distributed Storage
- Map/Reduce – Distributed Computing

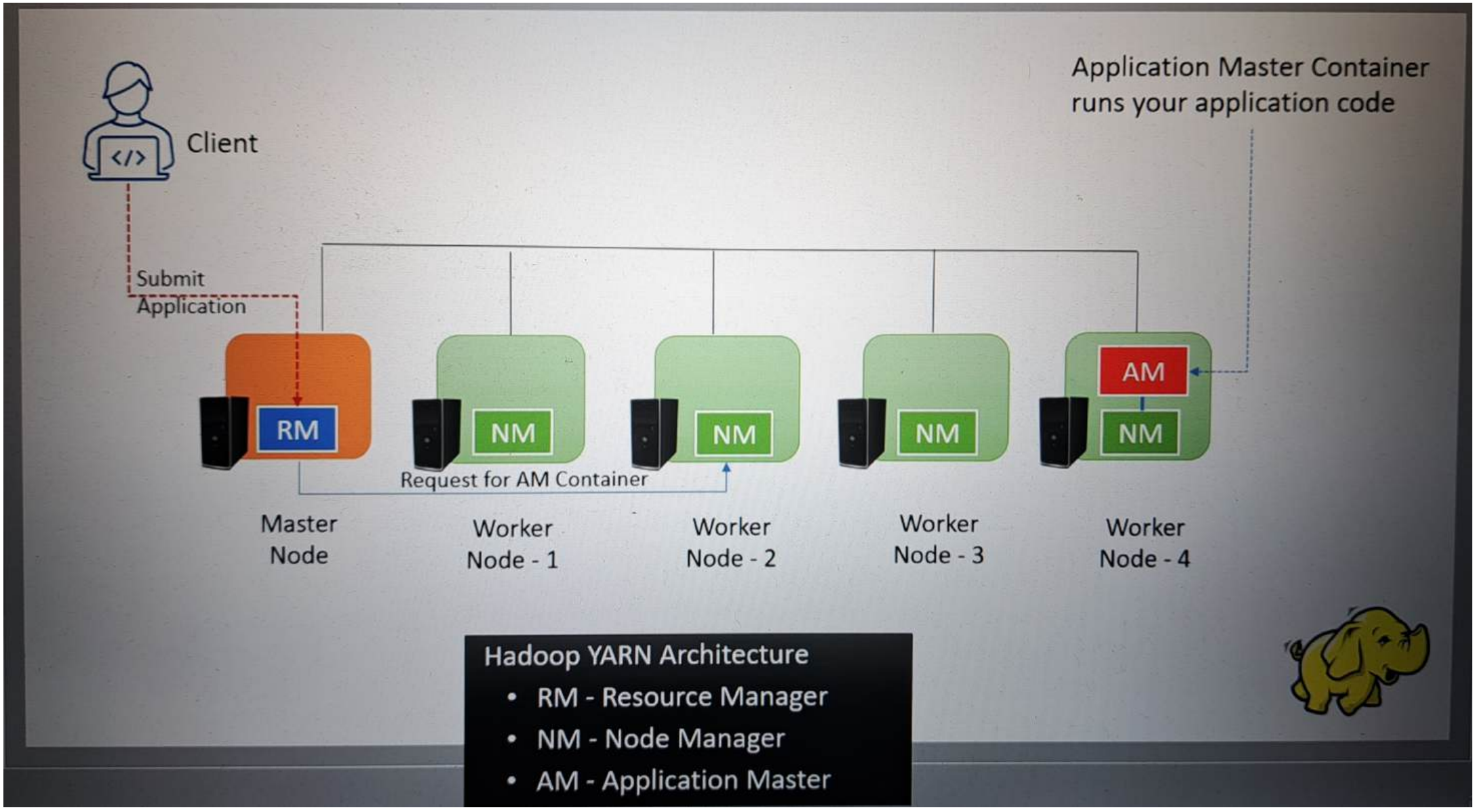


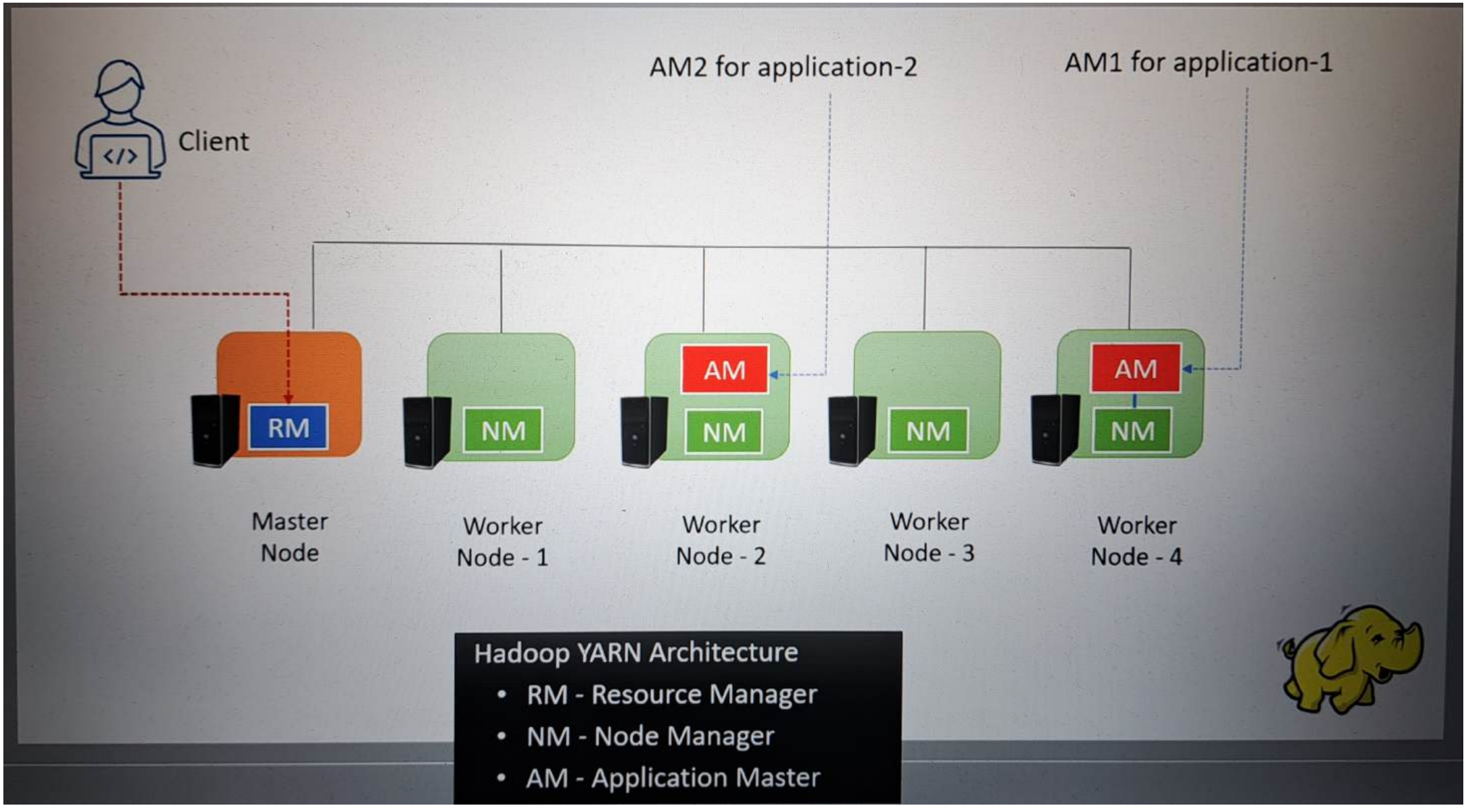
YARN – Yet Another Resource Manager

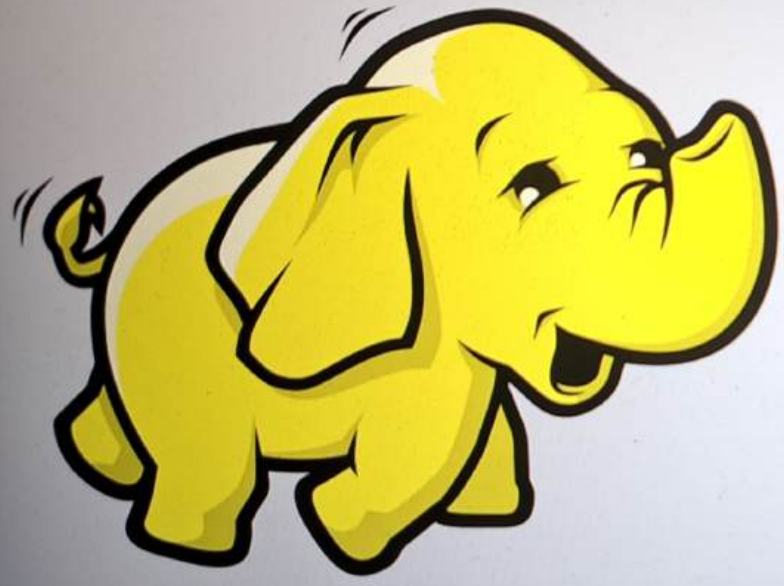
- Hadoop cluster Operating System
- Popularly known as Hadoop Cluster Resource Manager
- Has three main components
  - RM - Resource Manager
  - NM - Node Manager
  - AM - Application Master

udemy


Client
Start AM Container
RM
NM
NM
NM
NM
Master Node
Worker Node - 1
Worker Node - 2
Worker Node - 3
Worker Node - 4


Client
Application Master Container runs your application code
Submit Application
RM
NM
NM
NM
AM
NM
Request for AM Container
Master Node
Worker Node - 1
Worker Node - 2
Worker Node - 3
Worker Node - 4
Hadoop YARN Architecture
• RM - Resource Manager
• NM - Node Manager
• AM - Application Master

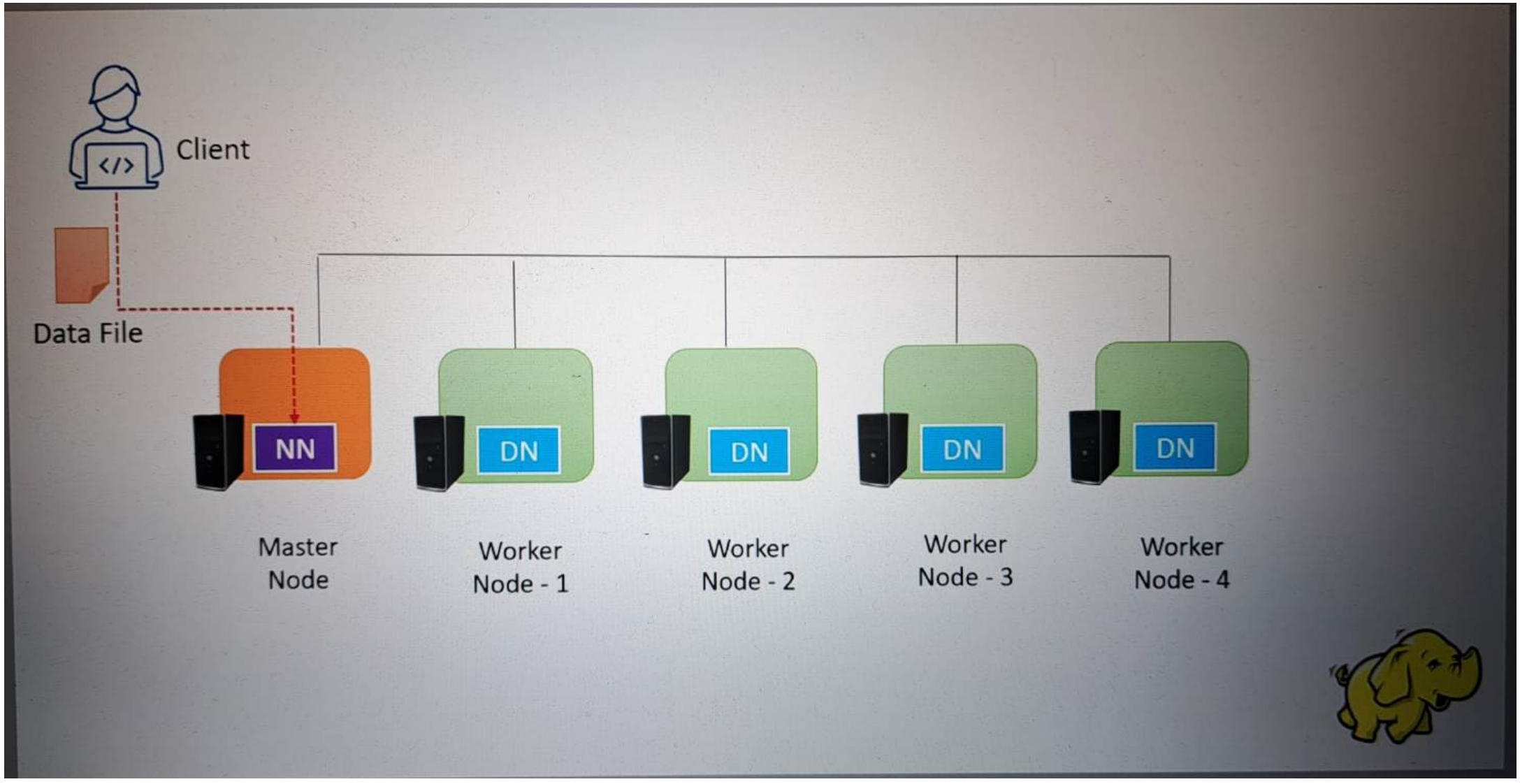

AM2 for application-2
AM1 for application-1
Client
RM
NM
AM
NM
NM
AM
NM
Master Node
Worker Node - 1
Worker Node - 2
Worker Node - 3
Worker Node - 4
Hadoop YARN Architecture
• RM - Resource Manager
• NM - Node Manager
• AM - Application Master

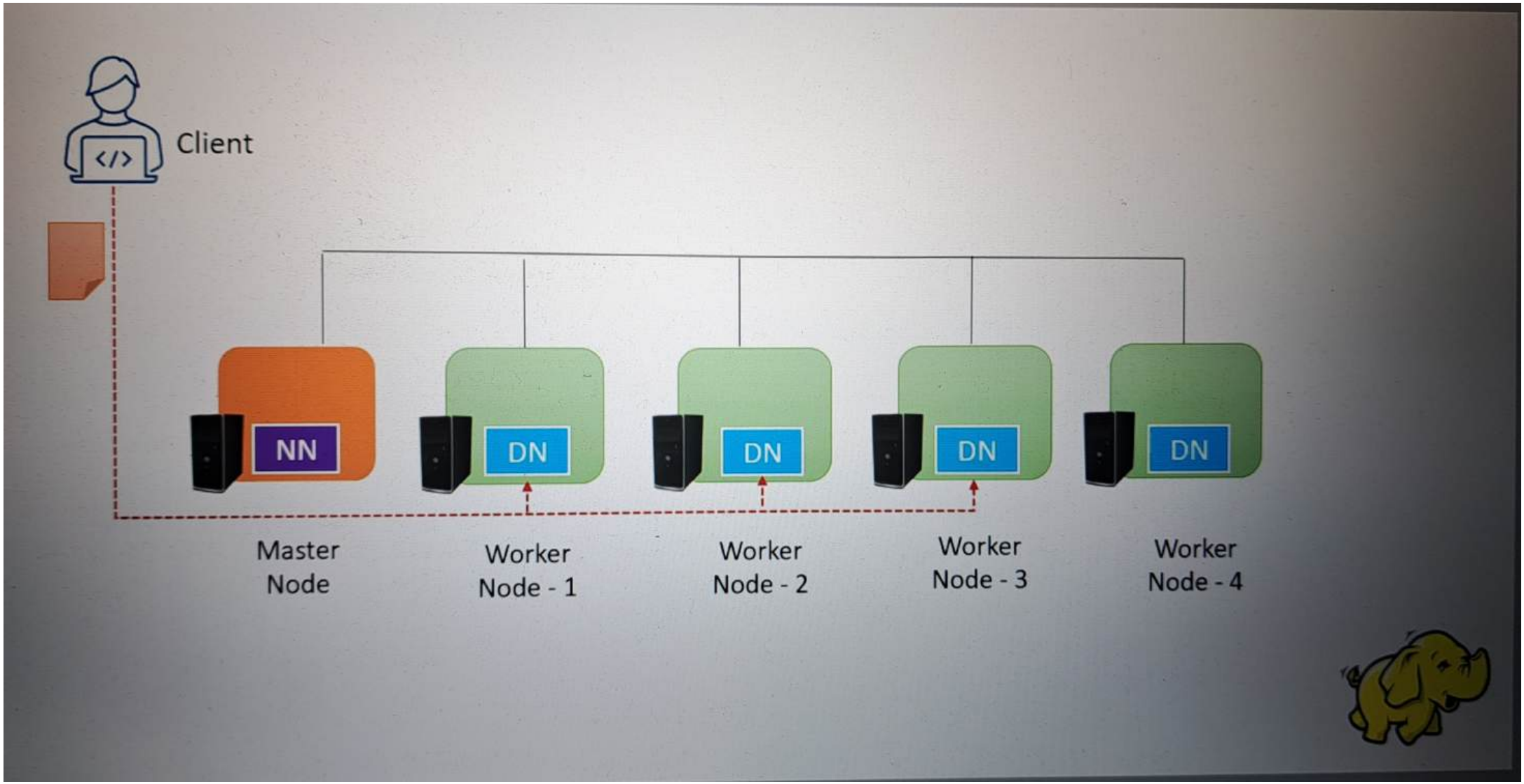


HDFS – Hadoop Distributed File System

- Distributed Storage on Hadoop Cluster
- Has two main components
  - NN – Name Node
  - DN – Data Node

udemy


Client
Data File
NN
DN
DN
DN
DN
Master Node
Worker Node - 1
Worker Node - 2
Worker Node - 3
Worker Node - 4

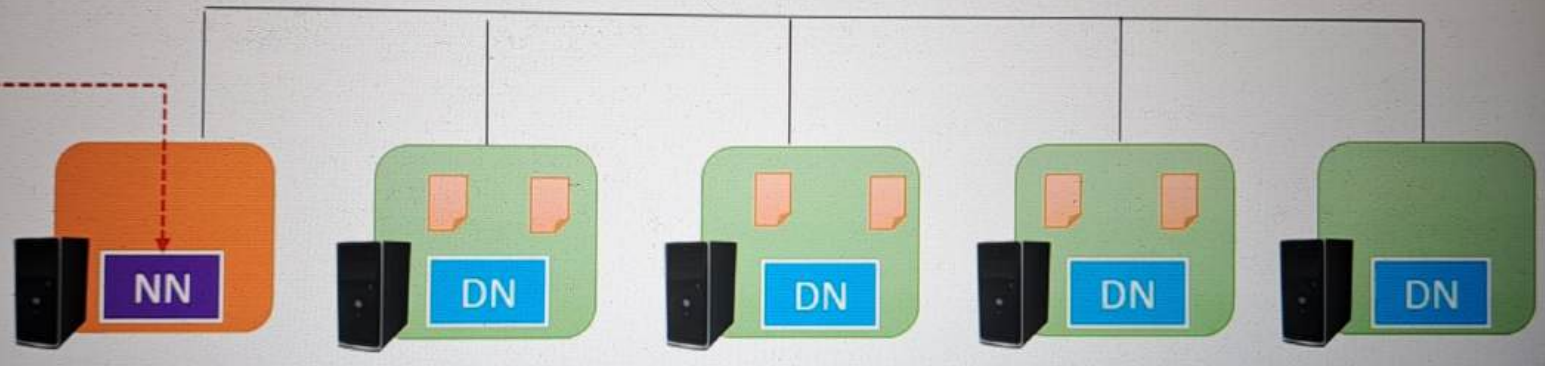

Client
NN
DN
DN
DN
DN
Master Node
Worker Node - 1
Worker Node - 2
Worker Node - 3
Worker Node - 4

File Metadata stored at Name Node

- File name
- Directory location
- File Size
- File blocks, block ID, block sequence, block location

File Metadata stored at Name Node

- File name
- Directory location
- File Size
- File blocks, block ID, block sequence, block location



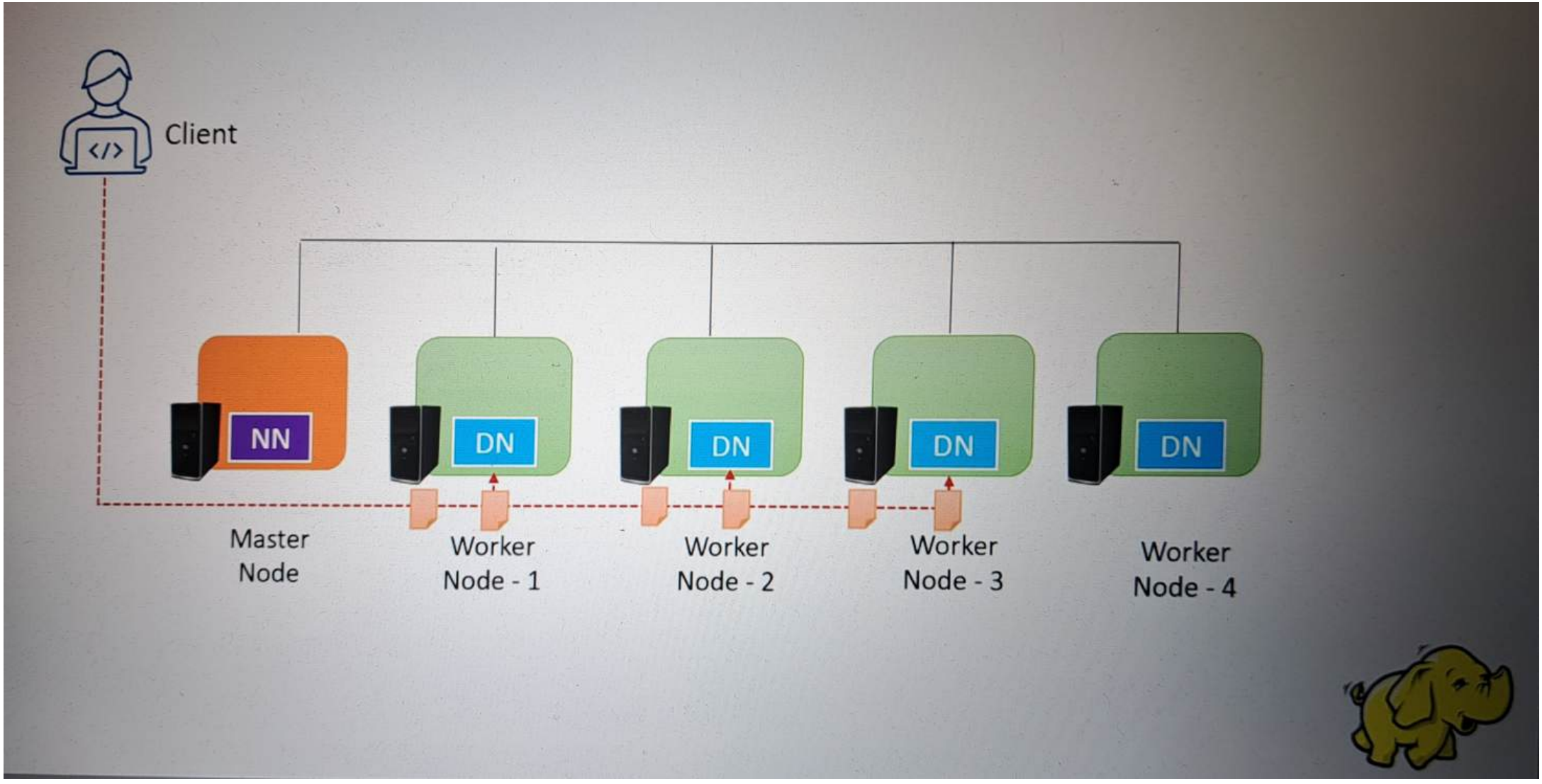
Master Node

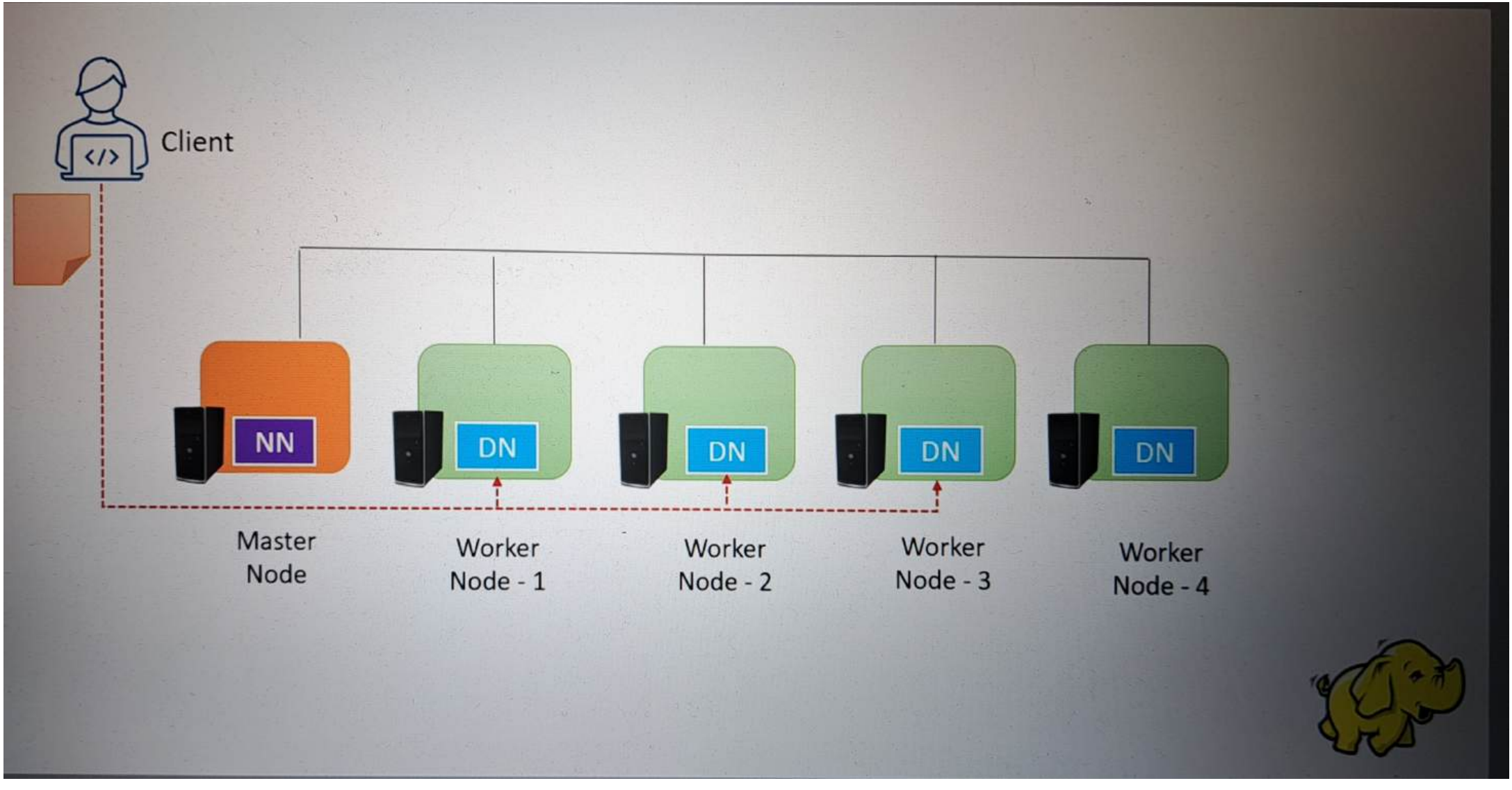
Worker Node - 1

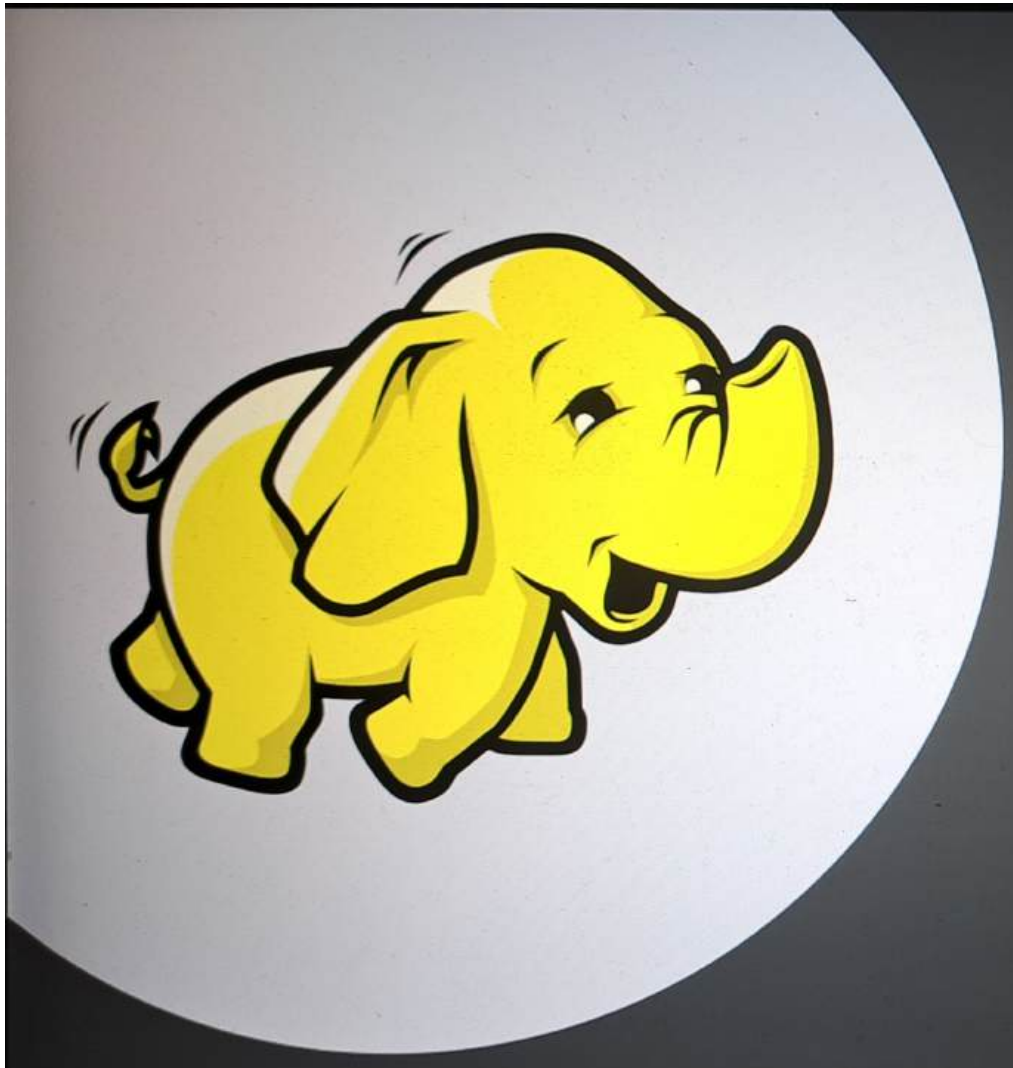
Worker Node - 2

Worker Node - 3

Worker Node - 4


Client
NN
DN
DN
DN
DN
Master Node
Worker Node - 1
Worker Node - 2
Worker Node - 3
Worker Node - 4


Client
NN
DN
DN
DN
DN
Master Node
Worker Node - 1
Worker Node - 2
Worker Node - 3
Worker Node - 4



M/R – Hadoop Map Reduce Framework

- Map Reduce is
  - Programing Model
  - Programing Framework

Udemy

Problem Statement

Count the lines in the given file

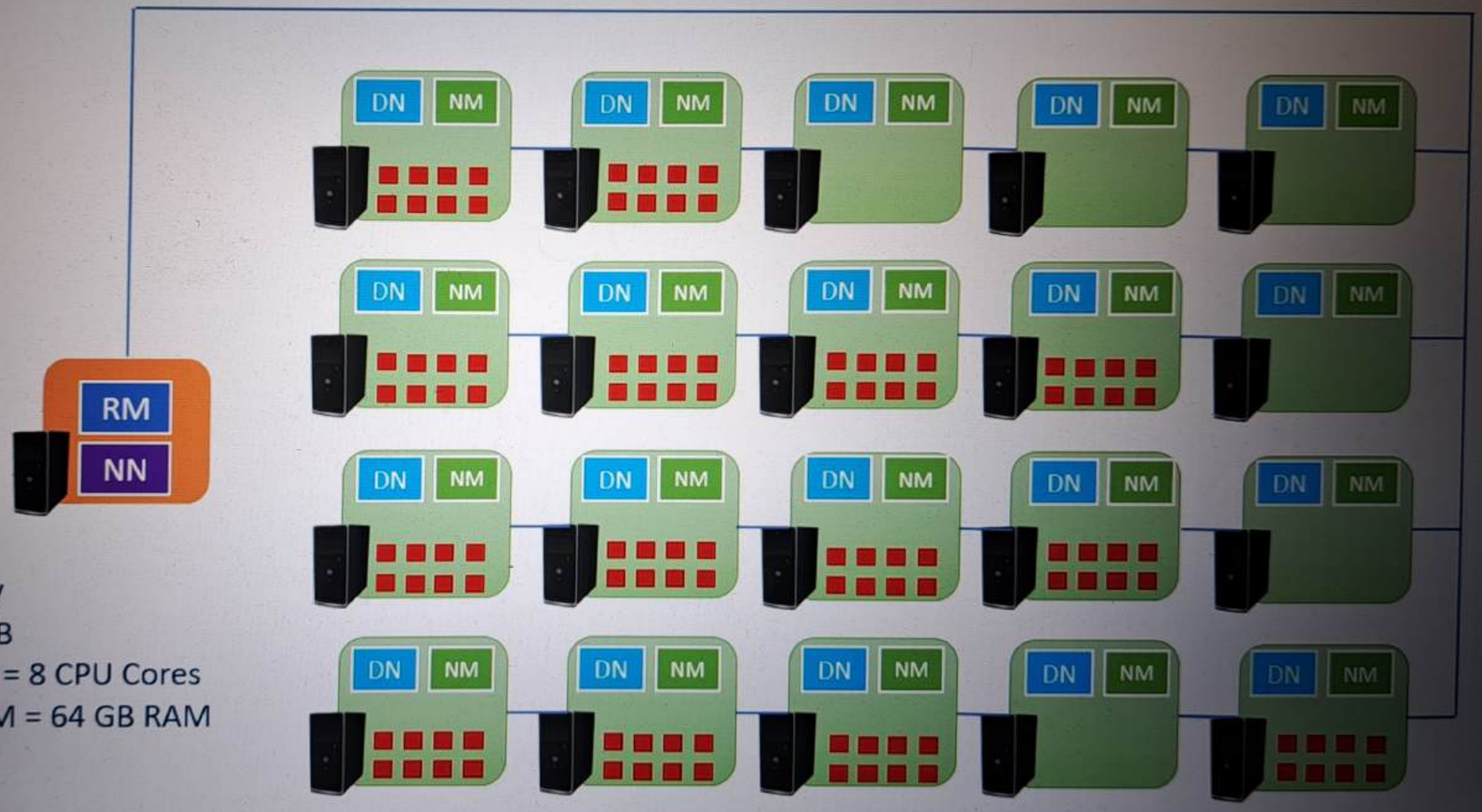
Solution Pseudocode

```
open file as f_hd
    for each t_line in f_hd.get_line()
        n_count = n_count + 1
close f_hd
print n_count
```

Challenges

1. Storage capacity
2. Processing time

Data File
Format: CSV
Size: 20 TB



Node Capacity

4 x 2 TB = 8 TB

4 x Dual Core = 8 CPU Cores

4 x 16 GB RAM = 64 GB RAM

Problem Statement

Count the lines in the given file

Distributed Solution Pseudocode

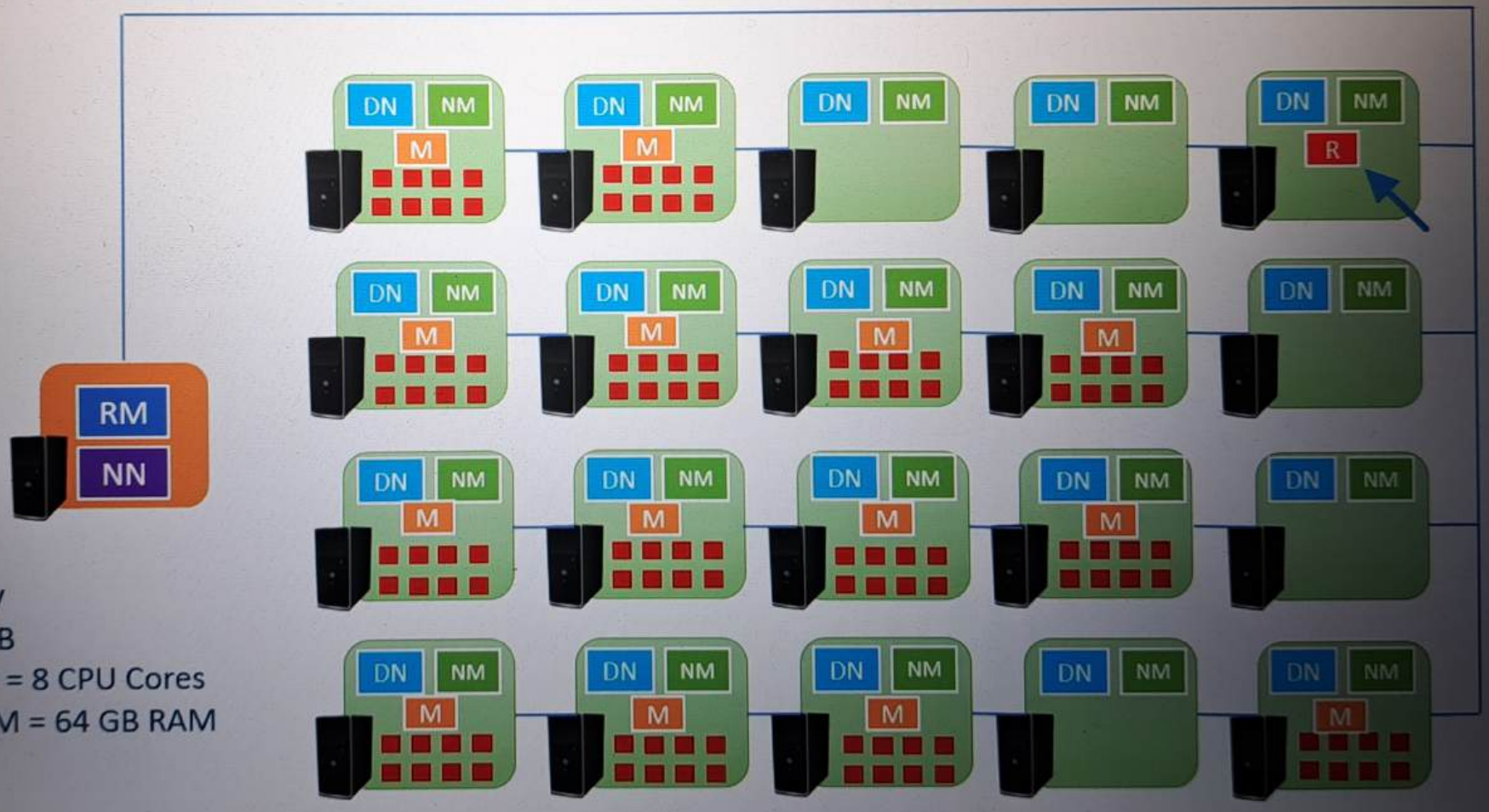
```
def map(file_block):
    open file_block as fb_hd
    for each t_line in fb_hd.get_line()
        n_count = n_count + 1

    close fb_hd
    return n_count
```

```
def reduce(list_counts):
    for each cnt in list_counts
        total_count = total_count + cnt

print total_count
```

Data File
Format: CSV
Size: 20 TB

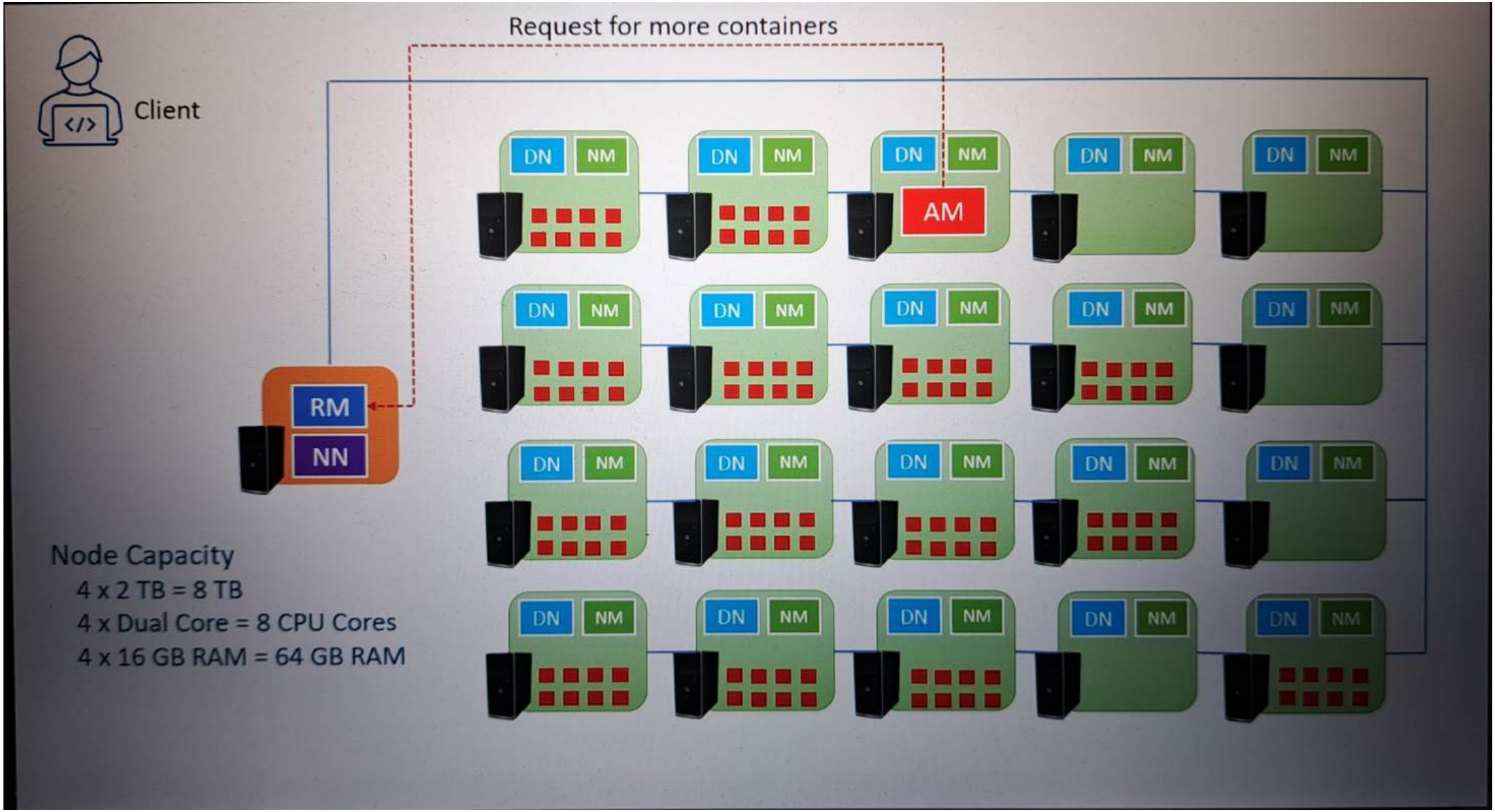


Node Capacity

4 x 2 TB = 8 TB

4 x Dual Core = 8 CPU Cores

4 x 16 GB RAM = 64 GB RAM


Request for more containers
Client
DN
NM
AM
RM
NN
Node Capacity
4 x 2 TB = 8 TB
4 x Dual Core = 8 CPU Cores
4 x 16 GB RAM = 64 GB RAM

Node Capacity

4 x 2 TB = 8 TB

4 x Dual Core = 8 CPU Cores

4 x 16 GB RAM = 64 GB RAM

Map Reduce Model

Implement logic in two functions

1. Map Function
   - Reads data block
   - Applies logic at block level
   - Map output is sent to Reduce
2. Reduce Function
   - Receives Map output
   - Consolidates the results

Hadoop M/R framework implement the map-reduce model.

- YARN manages resource allocation
- HDFS manages data blocks

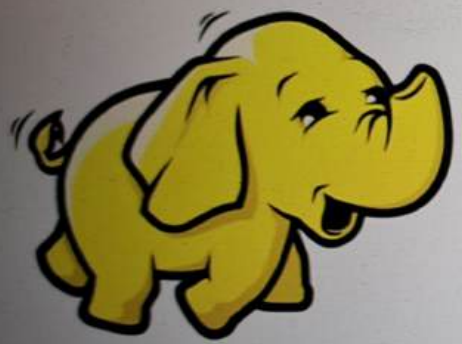
1. Data Collection and Ingestion
2. Data Storage and Management
3. Data Processing and Transformation
4. Data Access and Retrieval

1. Google File System – 2003

https://ai.google/research/pubs/pub51

2. MapReduce – 2004

https://ai.google/research/pubs/pub62



HADOOP

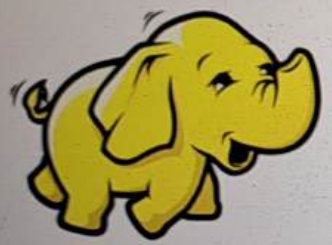
1. HDFS - Hadoop Distributed File System
2. Hadoop MapReduce

Hadoop Database

Hive capabilities

1. Create
    1. Databases
    2. Tables
    3. Views
2. Run SQL Queries

udemy



HADOOP

1. Performance
2. Ease of development
3. Language support
4. Storage
5. Resource Management

Entering Apache Spark

Advantages over Hadoop

1. Performance
   - 10 to 100 times faster than Hadoop M/R
2. Ease of development
   - Spark SQL
   - High performance SQL Engine
   - Composable Function API
3. Language support
   - Java, Scala, Python and R
4. Storage
   - HDFS Storage
   - Cloud Storage
5. Resource Management
   - YARN, Mesos, Kubernetes

APACHE Spark™

Runs in two setups

1. With Hadoop (Data Lake)
2. Without Hadoop (Lakehouse)

Spark and Databricks Cloud

Spark Solutions

1. Data Lake – On Hadoop
2. Lakehouse – On Cloud

APACHE Spark™

+

Databricks

udemy